Reuse Before You Retrieve: Diagnosing Headroom and Complementarity for Test-Time Augmentation of Embodied Multimodal Policies

2026. 11. 03

Seonyeong Jeong

Covered Papers

Reuse Before You Retrieve: Diagnosing Headroom and Complementarity for Test-Time Augmentation of Embodied Multimodal Policies (ECCV Workshop 2026)

Yuhwan Jeong and Kuk-Jin Yoon

VLA, Test-Time Augmentation, Recoverable Headroom, Retrieval Complementarity

Introduction

Test-Time Augmentation

- 새로운 task/environment/observation condition에서 frozen VLA policy의 실패 발생
- 환경마다 retraining하는 비용을 줄이기 위해 test-time 성능 개선 방법 활용

  → External demonstration retrieval 또는 additional sampling & selection 적용

Different Failure Modes

- 일부 stochastic rollout에서는 성공하지만 일관되게 수행하지 못하는 경우 ⇒ Repeated execution & selection이 유용할 것
- 반복 실행에서도 동일한 systematic failure가 발생하는 경우 ⇒ External behavioral prior 필요할 수 있음

➡ 유사한 single-rollout performance에서도 필요한 intervention이 달라질 가능성

Recoverable Headroom & Retrieval Complementarity

- **Recoverable headroom**: policy의 stochastic behavior 안에 이미 존재하는 개선 여지
- **Retrieval complementarity**: 외부 action prior가 기존 policy의 부족한 behavior를 보완할 가능성

➡ 두 diagnostic을 통해 기존 capability의 재사용과 external prior 도입의 기회 구분

Related Work

Retrieval-Augmented Robot Policies

- 유사한 demonstration, trajectory, sub-trajectory를 이용하여 robot behavior 개선
- 최근 VLA에서는 demonstration-derived memory와 trajectory context를 action generation에 통합
- 기존 연구의 초점: external experience의 표현/검색/통합 방법
  본 연구의 초점: frozen policy가 retrieved prior를 필요로 하는 조건/시점

Test-Time Compute and Selection

- 여러 candidate를 생성하고 선택하여 고정된 모델의 성능 개선
- 최근 연구들은 Action sampling, verifier, model-internal signal, adaptive critic 등의 접근 활용
- 본 연구는 selection rule뿐 아니라 selection이 활용할 수 있는 개선 여지를 측정
- Repeated rollout의 headroom과 demonstration fit을 통해 augmentation 필요성 진단

Method: Problem Setting

Frozen Stochastic Policy

- Initial state s_0에서 frozen VLA policy π로 rollout 생성
- Test-time intervention 동안 policy parameter 고정
- 동일 initial condition에서도 stochastic execution에 따라 outcome 변화

Recoverable Headroom

- **pass@1**: single-rollout success / **pass@N**: N개 rollout 중 하나 이상이 성공할 확률

$$\text{pass@1} = \mathbb{E}_\tau[\text{succ}(\tau)], \qquad \text{pass@}N = \mathbb{E}\left[\max_{i \le N} \text{succ}(\tau_i)\right]$$

- **H_N = pass@N – pass@1**: 동일 candidate set에서 얻을 수 있는 성능 향상 여지 → Recoverable Headroom
- 동일한 N개 rollout을 headroom 추정과 selector 평가에 사용

Shared Components

- 성공 demonstration의 observation과 action chunk로 library 구성: $\mathcal{D} = \{(o_j^d, a_j^d)\}_j$
- Selector는 observation, demonstration-fit probe는 action, retrieval은 demonstration segment 사용
- Selector와 retrieval은 frozen visual encoder 공유

Method: Measuring Retrieval Complementarity

Demonstration-Fit Error: Policy에서 sampling한 action distribution이 demonstrated behavior와 얼마나 가깝게 일치하는지 측정

- Demonstration observation을 입력하여 K개의 candidate action chunk sampling
- Demonstrated action과 가장 가까운 sample의 L2 distance 계산
- Demonstration state 전반의 평균으로 DFE 산출

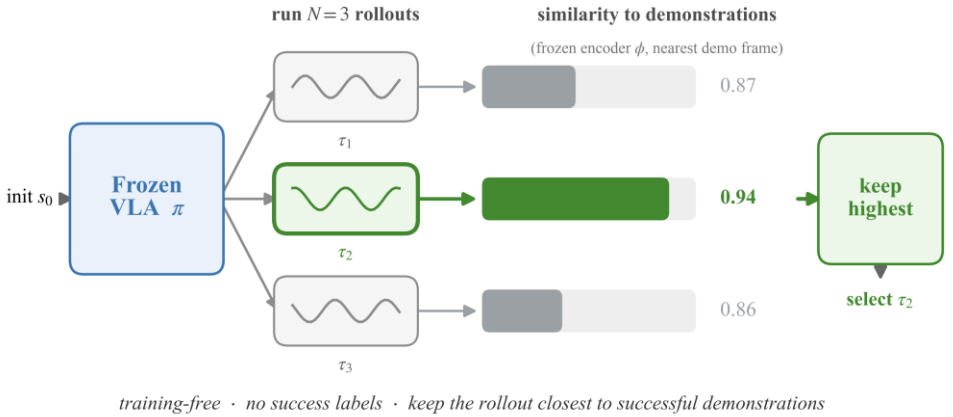
$$\mathrm{DFE} = \mathbb{E}_{(o^d, a^\star)\sim\mathcal{D}}\left[\min_{k\le K} \|a_k - a^\star\|_2\right], \qquad a_k \sim \pi(\cdot \mid o^d).$$

Action-Prior Mismatch

- **낮은 DFE**: policy sample과 demonstrated behavior 사이의 높은 overlap
- **높은 DFE**: 여러 sample 중에서도 demonstration과 가까운 action이 부족 = 큰 mismatch

➡ Policy가 demonstrated action prior를 얼마나 재현하는지 측정

Method: Episode-Level Retry Selection



Candidate Rollouts

- 동일 initial condition에서 서로 다른 seed로 N개의 rollout 실행
- 각 complete trajectory의 observation sequence를 scoring에 사용

Demonstration-Manifold Score: Shared encoder ϕ와 demonstration frame o_j^d 를 사용하여, 각 trajectory가 demonstration manifold에 평균적으로 얼마나 가까운지를 기준으로 score를 부여

- 각 observation과 가장 유사한 demonstration frame의 cosine similarity 계산
- 전체 trajectory의 평균 similarity를 rollout score로 사용

$$s(\tau) = \frac{1}{T}\sum_{t=1}^{T} \max_j \left\langle \phi(o_t), \phi(o_j^d) \right\rangle$$

- 가장 높은 score의 complete rollout 선택

$$\hat{\tau} = \arg\max_{i \le N} s(\tau_i)$$

Execution Setting and Cost

- Setting: 공통 initial state로 reset하거나 동일 system에서 parallel execution 수행
- Cost: N개 rollout 실행에 따른 N배의 execution overhead 발생

  Scoring은 frozen embedding의 nearest-neighbor comparison만 필요로 함

Method: Retrieval Augmentation

세 가지 component로 구성된 **event-schema prospective retrieval**로 retrieval 구현

Event-Start Frames

- Episode start, gripper open/close transition, 급격한 motion direction 변화로 event-start frame 식별
- 임의의 frame 대신 candidate sub-skill boundary에서 retrieval 수행

Event Matching

- Frozen image embedding, predicted action chunk, 이용 가능한 proprioceptive state 사용
- Dimension별 standardized distance의 weighted sum으로, 현재 event와 각 retrieval candidate 비교

Prospective Retrieval

- 현재 event와 matching한 demonstration의 다음 event action chunk 반환
- Retrieved action(a_{retr})과 sampling noise를 혼합하여 sampler warm-start
- a_{init}에서 시작하여 flow-matching ODE 적분

$$a_{\text{init}} = \alpha\, a_{\text{retr}} + (1-\alpha)\,\epsilon, \qquad \alpha \in [0,1],$$

Experimental Setup

Backbones

- LIBERO에서 fine-tuning된 frozen flow-matching VLA: **π_0, SmolVLA, $\pi_{0.5}$**
- Autoregressive architecture로의 확장 확인을 위한 **OpenVLA-7B** 추가 평가

Benchmarks

- **LIBERO**: 네 가지 suite (Spatial, Object, Goal, LIBERO-10), configuration별 500 × 3 rollouts
- **SimplerEnv-Bridge**: WidowX robot의 carrot, spoon, stack task, 24 × 3 rollouts
- **SimplerEnv coke-can**: Google robot, 25 initial states × 3 rollouts → OpenVLA-7B 평가
- **LIBERO-Occ**: occluded task variant, 250 × 2 rollouts

Experimental Setup

Implementation

- Shared Encoder ϕ: Frozen DINOv2 + SigLIP의 1,536-dimensional L2-normalized embedding
- Wrist-camera view 우선 사용, 없으면 third-person view 사용
- LIBERO demonstration library: task별 50개 demonstration
- 주요 설정: selector N = 3, retrieval α = 0.3
- DFE: 400 demonstration states, state별 K = 3 samples

Results: Selection and Retrieval Expose Different Opportunities

Main Comparison

- Selector는 모든 backbone과 12개 backbone × suite 조합에서 성능 개선
- Retrieval은 π_0에서만 네 suite 모두 positive gain 확인

Backbone	pass@1 (frozen)	+Retrieval	+Selector (ours)
π_0	64.2	70.6 (+6.4)	**85.2 (+21.0)**
SmolVLA	65.6	65.9 (+0.3)	**85.5 (+19.9)**
$\pi_{0.5}$	88.7	89.2 (+0.5)	**96.5 (+7.8)**

Why Retrieval Helps π_0 Consistently

- π_0와 SmolVLA의 평균 pass@1: 64.2%, 65.6%
- 동일 retrieval의 gain: 각각 +6.4 %p, +0.3 %p
- DFE: π_0 0.042, $\pi_{0.5}$ 0.021, SmolVLA 0.003

  → Demonstrated action prior와 **mismatch가 가장 큰 π_0**에서 **일관된 retrieval 이득**

Backbone	Mean pass@1	Spatial	Object	Goal	10
π_0	64.2	**+4.5±1.5**	**+9.8±2.5**	**+5.5±0.8**	**+6.0±2.3**
SmolVLA	65.6	+0.5±0.9	−0.2±2.0	+1.6±0.7	−0.9±1.6
$\pi_{0.5}$	88.7	−1.1±0.9	+0.6±0.6	+2.6±2.0	−0.1±2.8

Diagnostic Interpretation

- 비슷한 baseline success에서도 retrieval 효과 차이 발생
- DFE는 action-prior gap의 diagnostic으로 활용

  Retrieval gain에 대한 calibrated continuous predictor로 해석하지 않음

Results: Reuse and Retrieval Compose

Different Sources of Improvement

- Retrieval은 external action prior를 도입하여 **candidate distribution 수정**
- Selection은 해당 distribution에서 **생성된 stochastic rollout 중 하나를 선택**

➡ Retrieval로 warm-start한 N = 3 rollouts에 episode-level selector 적용해서 테스트

Combined Results

- π_0 평균 SR: pass@1 64.2%, retrieval 70.6%, selector 85.2%
  Retrieval + selector의 평균 SR: 87.5%

➡ 두 intervention 결합 시 평균적으로 추가 성능 향상

Suite	pass@1	+retrieval	+selector	retrieval+selector	combine − selector
spatial	63.6	68.1 (+4.5)	89.6 (+26.0)	92.4 (+28.8)	+2.8
object	77.1	86.9 (+9.8)	96.0 (+18.9)	97.2 (+20.1)	+1.2
goal	80.4	85.9 (+5.5)	97.6 (+17.2)	96.8 (+16.4)	−0.8
10	35.7	41.7 (+6.0)	57.6 (+21.9)	**63.8** (+28.1)	+6.2
mean	64.2	70.6 (+6.4)	85.2 (+21.0)	**87.5** (+23.3)	+2.4

Table 3: Retrieval and selection provide additional gains when combined on average. π_0 on LIBERO with n=500×3 rollouts. Values report success rate (%) with gains over pass@1 in parentheses. The final column reports the additional gain over selection alone.

Remaining Opportunity

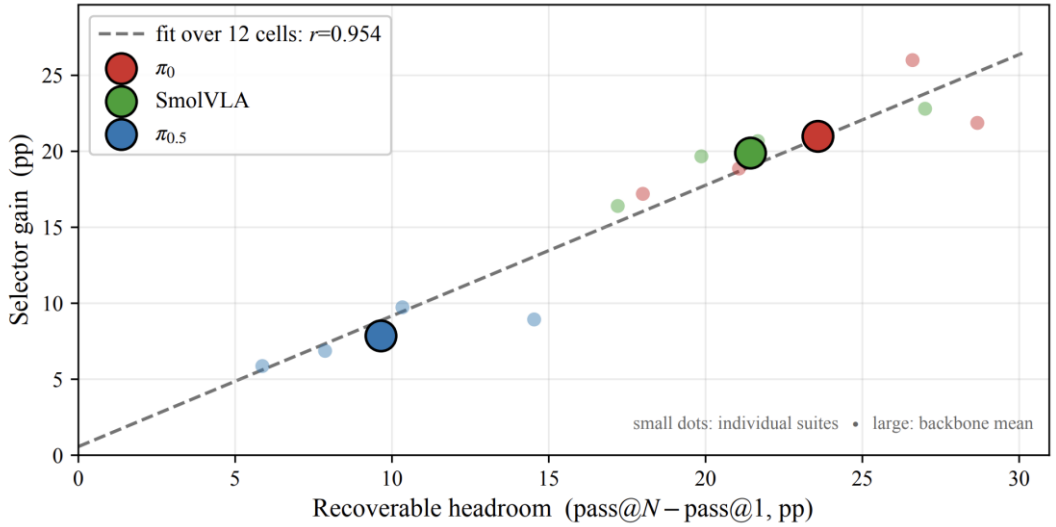
- Selection 이후 Remaining Opportunity가 얼마나 남아 있는지에 따라 추가 이득이 달라짐
- LIBERO-10: retrieval로 pass@3 ceiling 64.4%에서 71.0%로 증가 / 결합 방법은 selection 단독 대비 +6.2 %p 향상
- Goal: selection이 이미 97.6%에 도달하여 결합 방법에서 추가 향상 없음

➡ 두 intervention이 서로 다른 개선 기회에 작용한다는 해석과 일치

Results: The Headroom Relation

Gain Tracks Headroom

- 12개 backbone × suite 조합에서 평균 capture ratio gain/H_3 = 0.90
- Selector gain과 headroom의 correlation r = 0.954
  서로 다른 episode split에서도 r = 0.91 ± 0.04 유지



Selection, Not Resampling

- Rand-of-3는 pass@1 근처의 성능 유지
- Manifold-based selector는 세 backbone 모두 개선
- N = 2에서 N = 3으로 증가 시 추가 성능 향상
  → 더 나은 execution을 구분하는 ranking mechanism이 함께할 때 additional sample이 유용해짐

Headroom and Retrieval Complementarity

- Headroom과 retrieval gain의 correlation은 r = 0.32
- 큰 headroom이 있어도 retrieval의 이득은 작을 수 있음
- Policy 내부의 성공 가능성과 external prior의 보완성을 별도로 진단

Backbone	pass@1	rand-of-3	selector (N=2)	selector (N=3)	oracle pass@3
π_0	64.2	64.5	79.2	85.2	87.8
SmolVLA	65.6	65.3	81.0	85.5	87.0
$\pi_{0.5}$	88.7	89.0	94.8	96.5	98.3

Table 4: The gain comes from selecting additional samples rather than sampling alone. LIBERO 4-suite mean success rate (%). Randomly retaining one of three rollouts remains near pass@1, while manifold scoring recovers a large fraction of the available pass@3 headroom.

Results: Generalization Across Shifts

Cross-Simulator Transfer

- SimplerEnv-Bridge에서 selector의 overall gain +12.5 %p
  = Oracle headroom +18.1 %p의 약 69%를 끌어냄
- Retrieval은 평균 −0.9 %p로 domain에 따른 효과 변화 확인

Task	pass@1	+retrieval	+selector (ours)	oracle pass@N
carrot	34.7	30.6 (−4.2)	**41.7** (+6.9)	50.0
spoon	45.8	47.2 (+1.4)	**54.2** (+8.3)	62.5
stack	31.9	31.9 (−0.0)	**54.2** (+22.2)	54.2
overall	37.5	36.6 (−0.9)	**50.0** (+**12.5**)	55.6

Cross-Architecture Evaluation

- OpenVLA success: sample@1 0.40, selector 0.44, oracle pass@3 0.72
- 성공 candidate가 존재해도 ranking signal이 약하면 실제 gain 제한
- Observation view 등 조건이 달라 architecture만으로 원인 단정 불가

	Configuration	Success
baseline	greedy decoding	0.40
	sample@1 (T=1, 3-seed mean)	0.40 (+0.00)
per-step	re-rank (N=4)	0.36 (−0.04)
episode-level (ours)	selector (N=3)	**0.44** (+0.04)
	oracle pass@3	0.72

Cross-Condition Evaluation

- LIBERO-Occ에서 $\pi_{0.5}$의 성능 감소와 recoverable headroom 증가
- $\pi_{0.5}$ selector gain: clean +7.8 %p, occlusion +11.2 %p

	π_0			$\pi_{0.5}$		
Suite	pass@1	+retr	+sel	pass@1	+retr	+sel
spatial	61.6	+6.2	+**18.4**	80.8	+3.4	+**14.0**
object	81.0	+6.6	+**9.4**	74.0	+0.6	+**4.8**
goal	74.4	+2.8	+**9.6**	74.8	+2.0	+**12.0**
10	20.8	+1.8	+**10.4**	42.4	−2.6	+**14.0**
mean	59.5	+4.3	+**11.9**	68.0	+0.8	+**11.2**

➡ Observation condition에 따라 policy의 stochastic success profile이 바뀌며, selection이 활용할 수 있는 recoverable headroom 양의 변화

Discussion

Recoverable Headroom & Retrieval Complementarity

- Recoverable Headroom: repeated rollout에 추가적인 성공 behavior가 존재하는지 측정
- Retrieval Complementarity: retrieved action prior가 기존 policy behavior를 의미 있게 보완하는지 진단
- 두 개선 기회가 반드시 함께 나타나는 것은 아님

Characterizing Test-Time Augmentation

- Retryable execution에서 headroom을 추정하고 selection의 개선 여지 확인
- Demonstration fit을 통해 external action prior의 필요성 진단
- 두 개선 기회가 함께 존재하면 retrieval과 selection 결합 가능

Conditions for Test-Time Scaling

- Candidate set에 더 나은 outcome이 존재해야 함
- Selection signal이 해당 outcome을 식별할 수 있어야 함
- Headroom과 oracle–selector gap을 분리하여 평가할 필요

Limitations

Execution Model

- Repeated execution 또는 parallel execution이 가능한 setting에만 적용됨
- 한 번의 irreversible attempt에서의 성능 개선을 의미하지 않음! Retryable execution 조건에서 끌어낼 수 있는 capability
- Selection과 retrieval이 서로 다른 형태의 test-time cost 발생시킴 → equal-budget comparison 수행 X / 서로 다른 개선 기회 진단

Trajectory-Ranking Signal

- Ranking signal의 품질에 따라 headroom 회수 비율 변화 (Selection의 효과가 signal에 의존함)
- OpenVLA의 낮은 ranking 성능 원인을 분리하여 규명하지 못함 (LIBERO setting과 달라서)
- Global AUC는 within-episode candidate ranking의 직접 평가가 아님

Scope of Evidence

- DFE는 action representation과 sampling protocol에 의존
- Retrieval complementarity의 근거는 테스트한 backbone, retrieval mechanism에 한정
- 모든 실험은 simulation에서 수행

Idea & Discussion

Performance Degradation after Retrieval

- **Question**: Goal에서 retrieval을 결합하면 selector 단독보다 성능이 낮아지는 이유는 무엇일까?
- **Observation**: Selector 97.6%, retrieval + selector 96.8%로 0.8 %p 감소
- **Discussion**: 높은 성공률은 추가 향상 폭을 제한하지만, 결합 후 성능 하락의 원인까지 설명하지는 못함
- **Analysis**: 동일 initial state에서 retrieval 추가로 성공한 경우와 오히려 실패한 경우를 나누어 비교
- **Validation**: 여러 seed에서 반복 평가하여 0.8 %p의 성능 하락이 일관되게 나타나는지 확인

From Retryable Execution to a Single Irreversible Attempt

- **(Stated) Limitation**: 동일 initial state에서 N개의 complete rollout을 실제 실행한 뒤 선택하므로 reset 또는 parallel execution 필요
- **Question**: 실제 실행 기회가 한 번뿐이라면 candidate를 어떻게 비교하고 선택할 수 있을까?
- **Discussion**: 현재 score는 실행 중 얻은 observation sequence를 사용하므로, 실행 전에 계산할 수 없음
- **Possible Direction**: World model 또는 simulator에서 candidate rollout을 예측/평가한 뒤, 선택한 행동을 실제 환경에서 실행

Thank You